E-TİCARET SATIŞ ANALİZ SİSTEMİ

Ubuntu Sanal Makine | Docker Compose | PostgreSQL | Python | Grafana

Bu rehber, ham sipariş verilerini PostgreSQL üzerinde tutan, Python ile hesaplayıp ikinci tabloya yazan ve Grafana ile görselleştiren sistemi baştan sona kurmak içindir. Önceden kurduğunuz proje klasöründe de uygulanabilir.

1. Mimari ve hedef

```
Örnek siparişler -> PostgreSQL: orders
                         | SELECT
                         v
                    Python: app.py
                         | hesaplama + INSERT
                         v
               PostgreSQL: daily_sales_summary
                         | SQL sorgusu
                         v
                  Grafana Dashboard
```

Hesaplanacak ölçüler: günlük ciro, satılan ürün adedi, sipariş sayısı ve ortalama sipariş tutarı. Grafana hesaplamayı değil, sonuçların sorgulanmasını ve görselleştirilmesini üstlenir.

2. Ön kontroller ve klasörler

```
docker --version
docker compose version
sudo systemctl is-active docker
# Docker durmuşsa:
sudo systemctl start docker
sudo systemctl enable docker
mkdir -p ~/ecommerce-analytics/{postgres,python,grafana}
cd ~/ecommerce-analytics
touch docker-compose.yml postgres/init.sql python/app.py \
      python/Dockerfile python/requirements.txt
```

Docker servisinin bulunamadığı hatası alırsanız önce Docker Engine kurulumunu düzeltin. Bundan sonraki tüm komutları ~/ecommerce-analytics dizininden çalıştırın.

3. PostgreSQL başlangıç SQL dosyası

postgres/init.sql içeriği:

```
CREATE TABLE IF NOT EXISTS orders (
    order_id SERIAL PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    unit_price NUMERIC(12,2) NOT NULL CHECK (unit_price >= 0),
    order_date DATE NOT NULL
);

CREATE TABLE IF NOT EXISTS daily_sales_summary (
    sale_date DATE PRIMARY KEY,
    total_revenue NUMERIC(14,2) NOT NULL,
    total_quantity INTEGER NOT NULL,
    order_count INTEGER NOT NULL,
    average_order_value NUMERIC(14,2) NOT NULL
);
```

```
INSERT INTO orders (product_name, quantity, unit_price, order_date)
VALUES
('Laptop', 2, 25000, '2026-10-20'),
('Mouse', 5, 500, '2026-10-20'),
('Klavye', 3, 1500, '2026-10-21'),
('Monitor', 2, 8000, '2026-10-21'),
('Kulaklik', 4, 1200, '2026-10-22'),
('Laptop', 1, 25000, '2026-10-22'),
('Mouse', 8, 500, '2026-10-23'),
('Monitor', 1, 8000, '2026-10-23'),
('Klavye', 6, 1500, '2026-10-24'),
('Kulaklik', 3, 1200, '2026-10-24');
```

Önemli: init.sql yalnızca PostgreSQL veri dizini ilk oluşturulduğunda otomatik çalışır. Daha önce 10 sipariş eklediyseniz bu INSERT bölümünü yeniden manuel çalıştırmayın; aksi halde mükerrer kayıtlar oluşur.

4. Python bağımlılığı ve Dockerfile

python/requirements.txt

```
psycopg2-binary==2.9.10
```

python/Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
CMD ["python", "app.py"]
```

5. Python veri işleme algoritması

python/app.py dosyasının tamamı:

```
import os
from collections import defaultdict
from decimal import Decimal
import psycopg2

conn = psycopg2.connect(
    host=os.getenv("DB_HOST", "postgres"),
    database=os.getenv("DB_NAME", "ecommerce_db"),
    user=os.getenv("DB_USER", "ecommerce_user"),
    password=os.environ["DB_PASSWORD"],
    port=5432
)

try:
    with conn:
        with conn.cursor() as cursor:
            cursor.execute("""
                SELECT order_id, quantity, unit_price, order_date
                FROM orders
            """)
            orders = cursor.fetchall()
            daily_data = defaultdict(lambda: {
                "revenue": Decimal("0"),
                "quantity": 0,
                "order_count": 0
            })
            for order_id, quantity, unit_price, order_date in orders:
                daily_data[order_date]["revenue"] += quantity * unit_price
                daily_data[order_date]["quantity"] += quantity
                daily_data[order_date]["order_count"] += 1

            # Tek transaction: mevcut özetleri güncel veriden yeniden üret.
```

```
            cursor.execute("DELETE FROM daily_sales_summary")
            for date, data in daily_data.items():
                average = (
                    data["revenue"] / data["order_count"]
                ).quantize(Decimal("0.01"))
                cursor.execute("""
                    INSERT INTO daily_sales_summary (
                        sale_date, total_revenue, total_quantity,
                        order_count, average_order_value
                    ) VALUES (%s, %s, %s, %s, %s)
                """, (
                    date, data["revenue"], data["quantity"],
                    data["order_count"], average
                ))
            print(f"{len(orders)} sipariş okundu.")
            print(f"{len(daily_data)} günlük özet hesaplandı.")
finally:
    conn.close()
```

Decimal parasal hesaplama doğruluğunu korur. Her çalışmada özet tabloyu yeniden oluşturmak, değiştirilmiş veya silinmiş siparişleri de hesaba katar. İşlemler bir transaction içindedir; hata oluşursa veritabanı değişiklikleri geri alınır.

6. Docker Compose - tüm servisler

docker-compose.yml dosyasının tamamı:

```
services:
  postgres:
    image: postgres:17
    container_name: ecommerce-postgres
    restart: unless-stopped
    environment:
      POSTGRES_USER: ecommerce_user
      POSTGRES_PASSWORD: ecommerce123
      POSTGRES_DB: ecommerce_db
    ports:
      - "127.0.0.1:5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./postgres/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ecommerce_user -d ecommerce_db"]
      interval: 5s
      timeout: 5s
      retries: 10

  python:
    build: ./python
    container_name: ecommerce-python
    environment:
      DB_HOST: postgres
      DB_NAME: ecommerce_db
      DB_USER: ecommerce_user
      DB_PASSWORD: ecommerce123
    depends_on:
      postgres:
        condition: service_healthy
    restart: "no"

  grafana:
    image: grafana/grafana:12.2.0
    container_name: ecommerce-grafana
    restart: unless-stopped
    environment:
      GF_SECURITY_ADMIN_USER: admin
      GF_SECURITY_ADMIN_PASSWORD: admin123
    ports:
      - "3000:3000"
```

```
    volumes:
      - grafana_data:/var/lib/grafana
    depends_on:
      postgres:
        condition: service_healthy

volumes:
  postgres_data:
  grafana_data:
```

Bu şifreler yalnızca eğitim içindir. Gerçek ortamda secret/env yönetimi, erişim kısıtları ve güçlü şifreler kullanın. PostgreSQL host portu yalnızca Ubuntu localhost adresine açılmıştır.

7. Çalıştırma ve doğrulama

```
cd ~/ecommerce-analytics
sudo docker compose config
sudo docker compose up -d --build
sudo docker compose ps -a
sudo docker compose logs python
```

Beklenen durum: PostgreSQL Up (healthy), Grafana Up, Python Exited (0). Python loglarında "10 sipariş okundu" ve "5 günlük özet hesaplandı" yazmalıdır.

```
sudo docker exec -it ecommerce-postgres \
  psql -U ecommerce_user -d ecommerce_db

SELECT COUNT(*) FROM orders;
SELECT * FROM daily_sales_summary ORDER BY sale_date;
\q
```

Tarih	Ciro (TL)	Ürün adedi	Sipariş	Ortalama (TL)
20 Ekim	52.500	7	2	26.250
21 Ekim	20.500	5	2	10.250
22 Ekim	29.800	5	2	14.900
23 Ekim	12.000	9	2	6.000
24 Ekim	12.600	9	2	6.300

8. Grafana giriş ve PostgreSQL bağlantısı

Ubuntu VM tarayıcısından http://localhost:3000 adresini açın. Giriş: admin / admin123 (ilk girişte şifre değiştirilebilir). Windows ana bilgisayardan erişim için VirtualBox NAT port yönlendirme veya uygun bridged ağ bağlantısı gerekir.

Grafana menüsünde Connections → Data sources → Add new data source → PostgreSQL yolunu izleyin:

Alan	Değer
Host URL	postgres:5432
Database	ecommerce_db
Username	ecommerce_user

Alan	Değer
Password	ecommerce123
TLS/SSL Mode	disable
PostgreSQL version	17

Save & test düğmesiyle bağlantıyı doğrulayın. Docker ağı içinden PostgreSQL sunucusunun adı "postgres"tir; bu yüzden Grafana tarafında localhost:5432 kullanmayın.

9. Grafana dashboard panelleri

Dashboards → New → New dashboard → Add visualization yolunu izleyin. PostgreSQL veri kaynağını seçin; SQL editöründe Code modunu kullanın. Dashboard zaman aralığını 20-25 Ekim 2026 yapın.

Panel A: Günlük Satış Geliri - Time series

```
SELECT sale_date AS "time", total_revenue AS "Günlük Ciro"
FROM daily_sales_summary
WHERE $__timeFilter(sale_date)
ORDER BY sale_date;
```

Panel B: Toplam Ciro - Stat (beklenen 127.400 TL)

```
SELECT COALESCE(SUM(total_revenue), 0) AS "Toplam Ciro"
FROM daily_sales_summary
WHERE $__timeFilter(sale_date);
```

Panel C: Sipariş Sayısı - Stat (beklenen 10)

```
SELECT COALESCE(SUM(order_count), 0) AS "Sipariş Sayısı"
FROM daily_sales_summary
WHERE $__timeFilter(sale_date);
```

Panel D: Günlük Satış Özeti - Table

```
SELECT sale_date AS "Tarih",
       total_revenue AS "Ciro",
       total_quantity AS "Ürün Adedi",
       order_count AS "Sipariş Sayısı",
       average_order_value AS "Ortalama Sipariş"
FROM daily_sales_summary
WHERE $__timeFilter(sale_date)
ORDER BY sale_date;
```

Panellerde para alanları için Standard options → Unit üzerinden TRY/Türk lirası biçimini seçebilirsiniz. Dashboard değişikliklerini kaydedin.

10. Uçtan uca test: yeni sipariş

```
sudo docker exec -it ecommerce-postgres \
  psql -U ecommerce_user -d ecommerce_db

INSERT INTO orders (product_name, quantity, unit_price, order_date)
VALUES ('Laptop', 3, 30000, '2026-10-24');
\q

sudo docker compose run --rm python

sudo docker exec -it ecommerce-postgres \
  psql -U ecommerce_user -d ecommerce_db
SELECT * FROM daily_sales_summary
WHERE sale_date = '2026-10-24';
\q
```

24 Ekim için beklenen yeni değerler: ciro 102.600 TL, ürün adedi 12, sipariş sayısı 3, ortalama sipariş tutarı 34.200 TL. Grafana dashboard'unu yenileyince güncel değerler görünür. Bu örnekte Python otomatik değil, manuel yeniden çalıştırılır.

11. Sorun giderme

Belirti	Kontrol / çözüm
Docker permission denied	sudo docker compose ... kullanın.
Connection refused	sudo docker compose ps ile postgres durumunu kontrol edin.
orders tablosu yok	init.sql çalıştı mı, doğru veritabanında mısınız?
orders boş	SELECT COUNT(*) FROM orders; kontrol edin.
Grafana bağlantı hatası	Host: postgres:5432, kullanıcı ve parola doğru mu?
Grafana grafik boş	20-25 Ekim 2026 zaman aralığını seçin.
Python Exited (0)	Normal: tek seferlik görev başarıyla tamamlandı.

```
sudo docker compose logs postgres
sudo docker compose logs python
sudo docker compose logs grafana
```

12. Günlük kullanım ve dikkat edilecekler

```
sudo docker compose up -d          # Başlat
sudo docker compose stop           # Durdur
sudo docker compose restart        # Yeniden başlat
sudo docker compose ps -a          # Durum
sudo docker compose run --rm python # Hesaplamayı yeniden yap
sudo docker compose build python   # Python image yeniden oluştur
sudo docker compose down           # Container ve ağı kaldır
```

Uyarı: docker compose down -v volume verilerini de siler. PostgreSQL kayıtları ve Grafana dashboard'ları kaybolabilir. Mevcut veritabanında başlangıç SQL dosyasını tekrar çalıştırmak aynı siparişleri yeniden ekleyebilir.

13. Tamamlama listesi

☐ Docker ve Compose çalışıyor

☐ PostgreSQL healthy

☐ orders tablosunda 10 başlangıç kaydı var

☐ Python okuyor, hesaplıyor ve sonuç tablosuna yazıyor

☐ Grafana PostgreSQL veri kaynağı testini geçiyor

☐ Dashboard dört panel içeriyor

☐ Yeni sipariş sonrası sonuçlar güncelleniyor

Sonuç: PostgreSQL → Python hesaplama → PostgreSQL sonuç tablosu → Grafana zinciri tamamlanmış olur. İleride otomatik çalıştırma, CSV/API verisi, alarm ve gelişmiş ürün analizleri eklenebilir.